

# DiracX developments: directions

**Alexandre F. Boyer** · CERN, on behalf of the DIRAC Consortium

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026

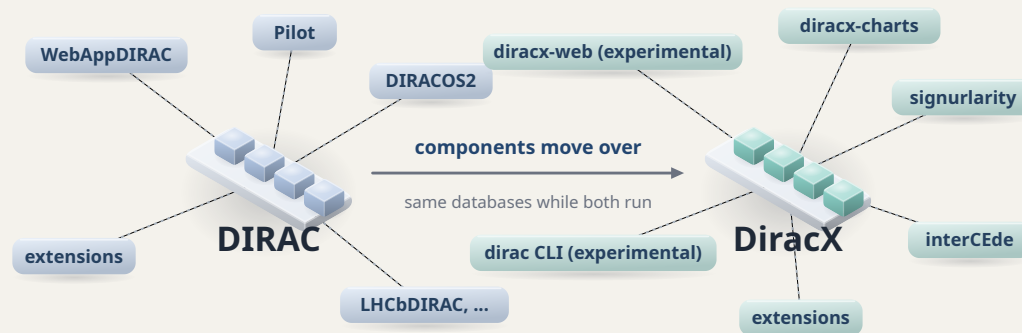


# | 1 • Where we are now

DIRAC and DiracX today



# Maintaining DIRAC, progressing on DiracX



**DIRAC** v9.1.21 · v9.0.27 · v8.0.85 · WebAppDIRAC 6.0.6 · DIRACOS2 2.63. **Patched, not developed:** security fixes, and performance fixes when needed.

**DiracX** v0.5.0 (29 Sep): authentication, jobs (search, status, submission), sandboxes, read-only resource status, a task framework. The web app (diracx-web 0.1.0-a11) and the [dirac](#) CLI are still experimental.

**Move to what exists today as soon as you can.** DIRAC only gets patches so that DiracX can progress. LHCb runs one service and one task in production; two other installations are certifying DiracX.

## | 2 · Evolving requirements, fruitful collaboration

New communities, new use cases, and two systems running side by side



# New communities, new use cases

## What we heard

- **CMS**, who joined this summer, has no DIRAC installation and runs its own workload management (HTCondor with GlideinWMS). It needs to hand work to a system we do not own, to separate taking requests from executing them, and to describe work in CWL.
- **GridPP**, contributing for a while, wants to deploy DiracX without Kubernetes, with multi-VO from the start. It is mostly interested in the WMS and the DMS.
- **CTAO, IN2P3, FCC, EGI, Belle II and IHEP** contribute too, several of them for a long time.

## DIRAC's values, in DiracX

Few dependencies: MySQL, OpenSearch

**Standard protocols: S3, OpenTelemetry, OIDC**

Several communities, one installation

**Multi-VO from the start**

Communities add their own pieces

**Extensions, with a reference extension**

Small installations on a few hosts

**Options that do not need Kubernetes**

# Two audiences, one project

## Communities on DIRAC today

They want **stability and a smooth transition**.

- Move **component by component**, over the same databases. No flag day.
- DIRAC keeps running, with **security and performance patches**.
- Your extension keeps working while you move.
- **The whole production chain is new**: production (workgraph), transformation and job (parcel). We plan it with you.

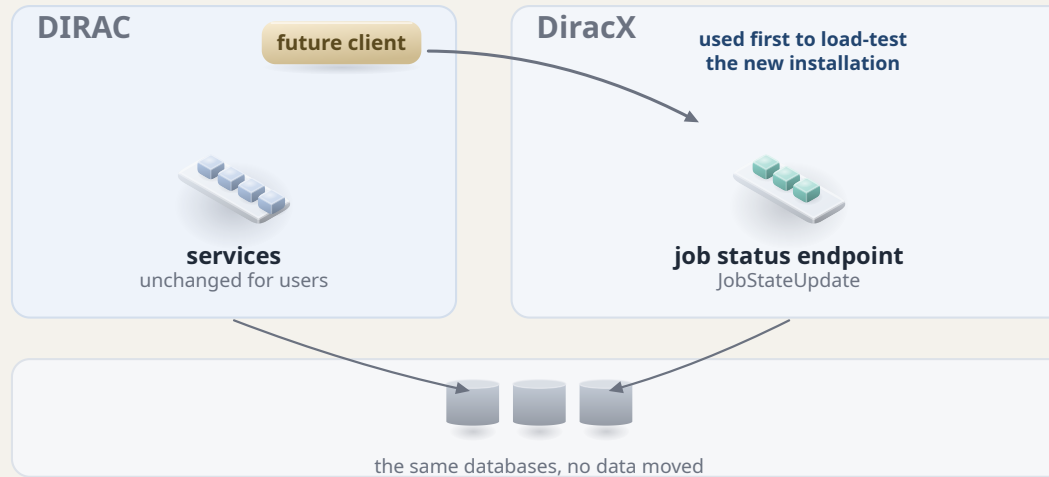
## Communities without DIRAC

They should **not feel DIRAC's legacy technical choices**.

- A **new data model** for transformations, not DIRAC's.
- **Standard protocols**, multi-VO from the start.
- **Their own execution** (HTCondor, for example) behind a compute backend.
- Their requirements go through **the same process** as everyone's.

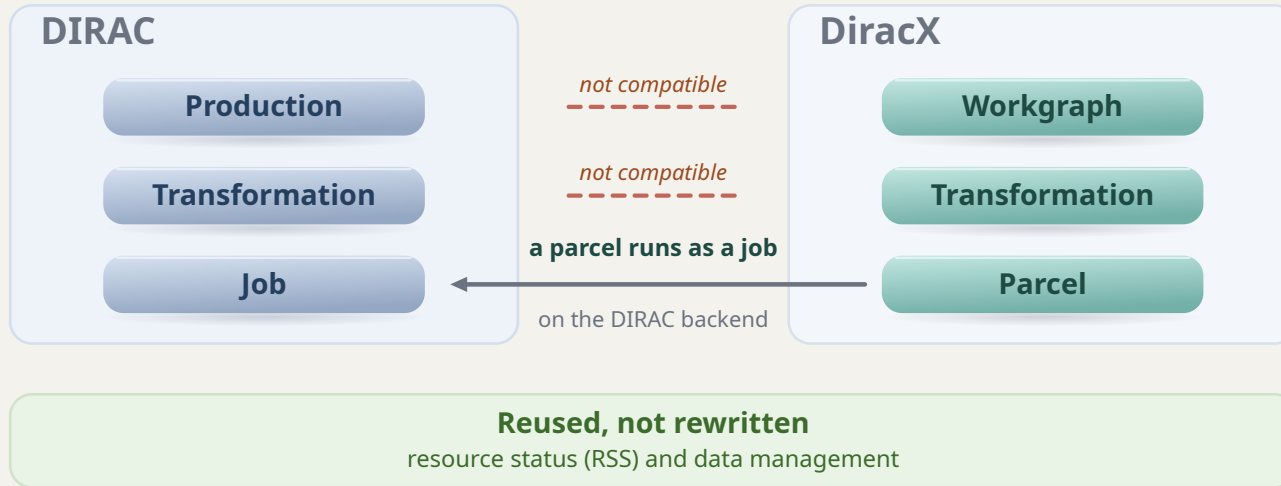
One project, one process, two paths. We respond to both as requirements arrive.

# Side by side, step 1: future clients



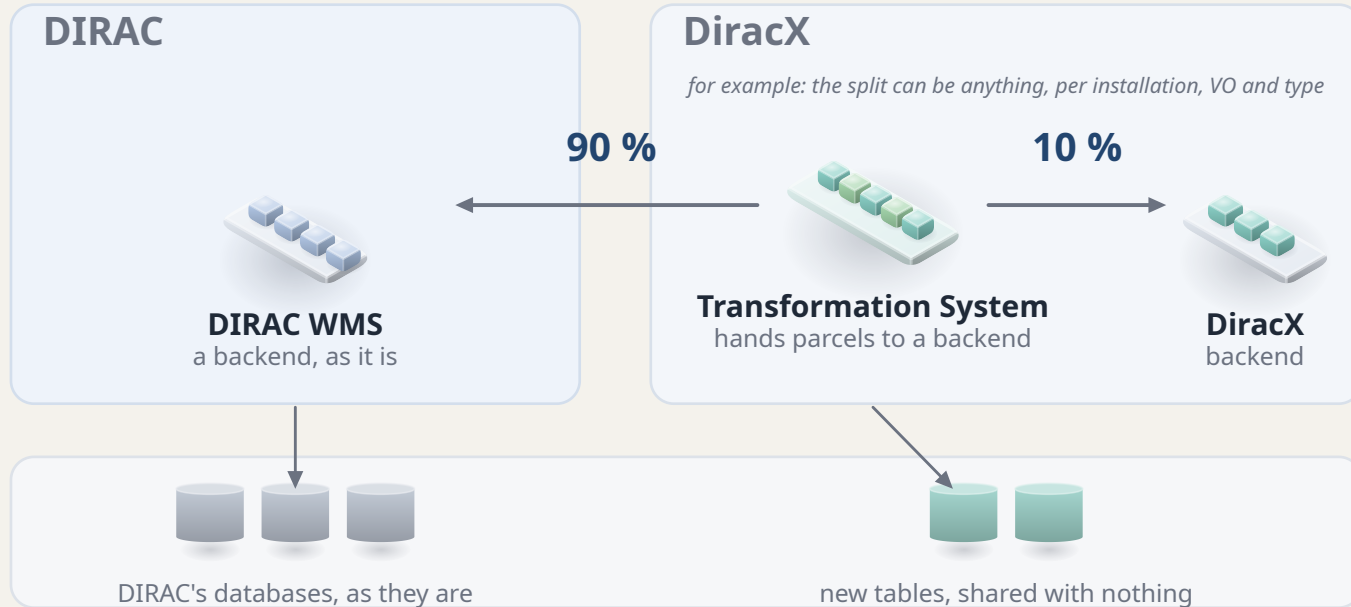
- Job status goes through DiracX over the **same databases, no data moved**: the first step of a deployment, to check the new system works as expected.
- **Jobs themselves are not carried over**. Parcels replace them.

# What changes for transformations



- **New concepts, no compatibility:** production and workgraph, old and new transformation, job and parcel do not map onto each other.
- **One bridge:** a parcel can run as a DIRAC job. **Resource status and data management are reused.**

# Side by side, step 2: a backend for each parcel



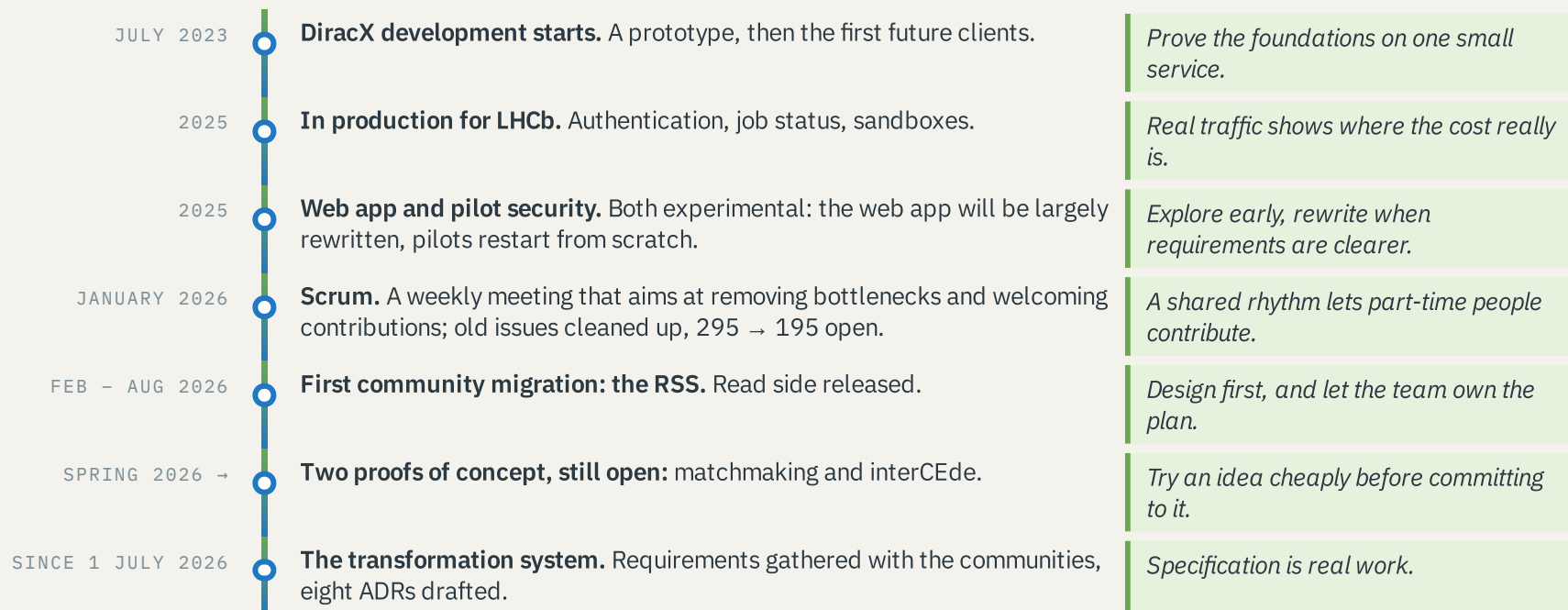
- Your **DIRAC WMS** keeps running the jobs. DiracX keeps the books, in new tables.
- **Move gradually and reversibly**: start at 100% on DIRAC, raise the DiracX share per installation, VO and type.

## 3 · Adapting quickly

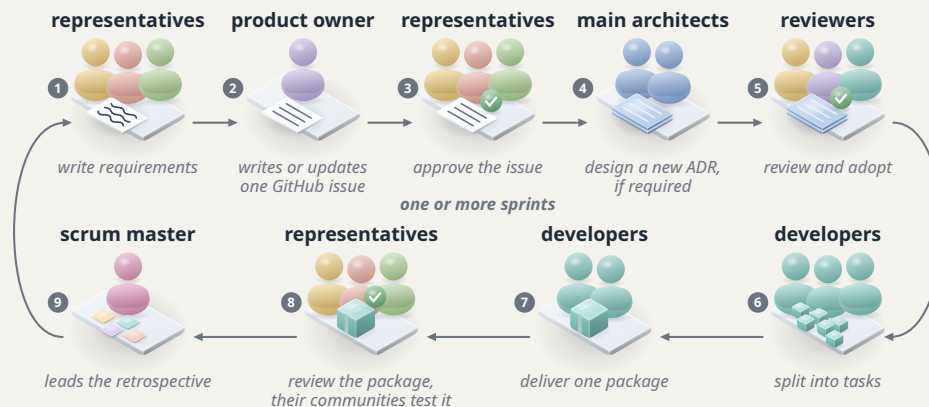
Responding to changing requirements, in small steps



# The journey so far



# The current process, open to change



- **Scrum since 21 January 2026:** two-week sprints; planning and refinement in one weekly meeting; a monthly meeting with installation administrators; hackathons.
- **Each community has representatives.** Design decisions are ADRs by a main architect. ADRs and roadmap are public; the board is not.
- **It is not fixed.** Like the roadmap, we adapt the process as we learn, and we are open to optimising it.

## | 4 · Results so far

Numbers since the last workshop



# Activity since the last workshop

| 17 SEP 2025 – 4 SEP 2026                | DIRAC STACK   | DIRACX STACK   | BOTH           |
|-----------------------------------------|---------------|----------------|----------------|
| Merged pull requests                    | 367           | 261            | 634            |
| People who merged one                   | 20            | 28             | 36             |
| People who gave a first review          | 12            | 11             | 17             |
| New contributors                        | 9             | 14             | 17             |
| Issues opened                           | 89            | 206            | 296            |
| Issues closed (completed / not planned) | 117 (76 / 41) | 224 (191 / 33) | 343 (268 / 75) |

DIRAC stack: DIRAC, WebAppDIRAC, DIRACOS2, Pilot. DiracX stack: diracx, diracx-charts, diracx-web, signurlarity, dirac-cwl, interCEde. Bots excluded; copies of one change on several release branches counted once. Issues to 2 Sep.



# Since Scrum: the backlog is shrinking

**295 → 195**

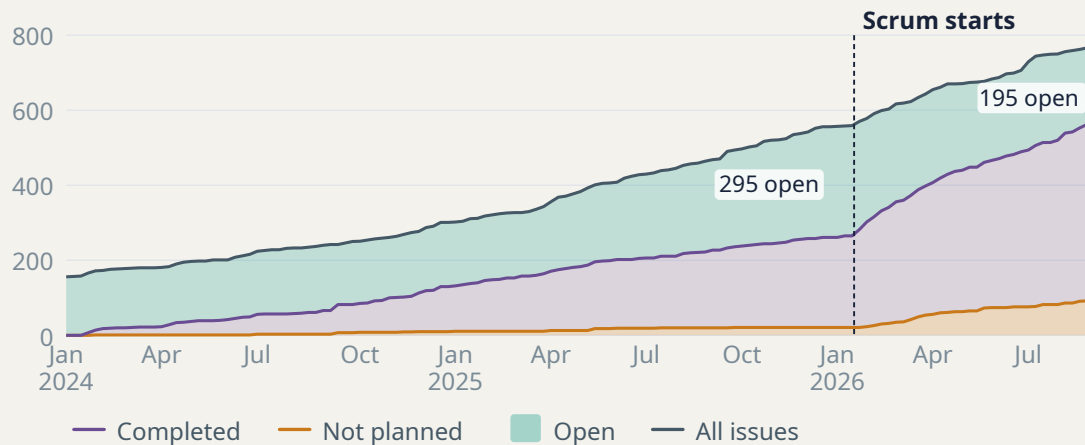
open issues, January 2026 to September

**10.6 → 36.6**

closures a month; 26.6 completed, 10.0 not planned or duplicate

**16.3 → 27.3**

issues opened a month



All issues in the DIRACGrid repositories, bots excluded. Part of the fall is triage.

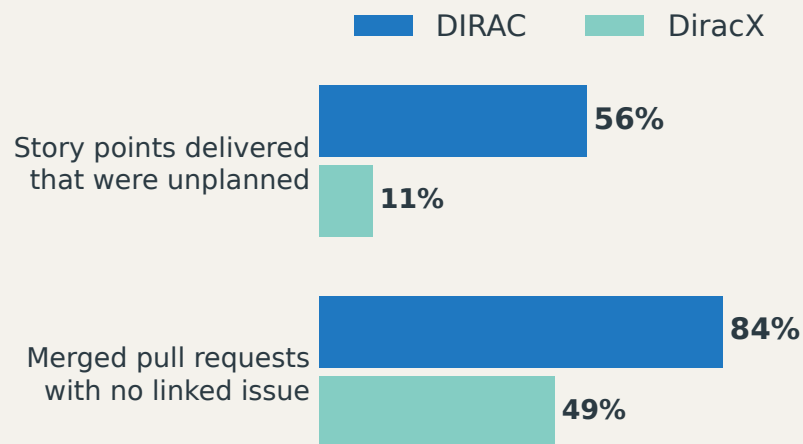
# Where an issue goes



*issues on the planning board, data to 4 September 2026*

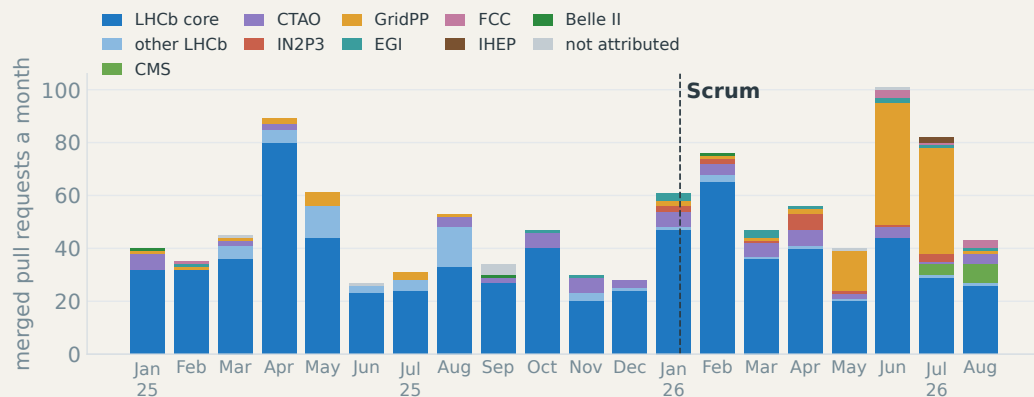
- Issues that move on do so quickly. The ones still waiting have waited far longer.
- 121 issues sit in triage, most since the January board sweep. 34 are "in progress" with a median of two months.

# Unplanned work lives in DIRAC



- Since January, **56% of the story points delivered on DIRAC were unplanned**, against 11% on DiracX.
- **84% of DIRAC pull requests have no linked issue**, against 49% on DiracX.
- Security and performance work arrives when it arrives. It is the cost of keeping DIRAC running, and why DIRAC only gets patches.

# More hands, from more communities

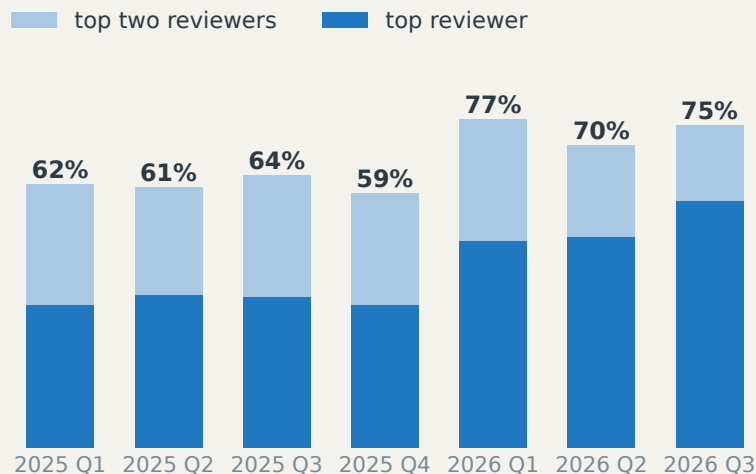


**36 people** merged a change since the last workshop; **17 were new**, from ten communities. Almost all are part-time, so effort varies from month to month.

Outside the five core developers: **36% of merged PRs, 28% of issues opened, 53% of story points.**

Read with care: about half of the 2026 rise in PRs is **one GridPP developer** hardening DIRAC's SQL. CMS merged 14 changes since July.

# The bottleneck is review

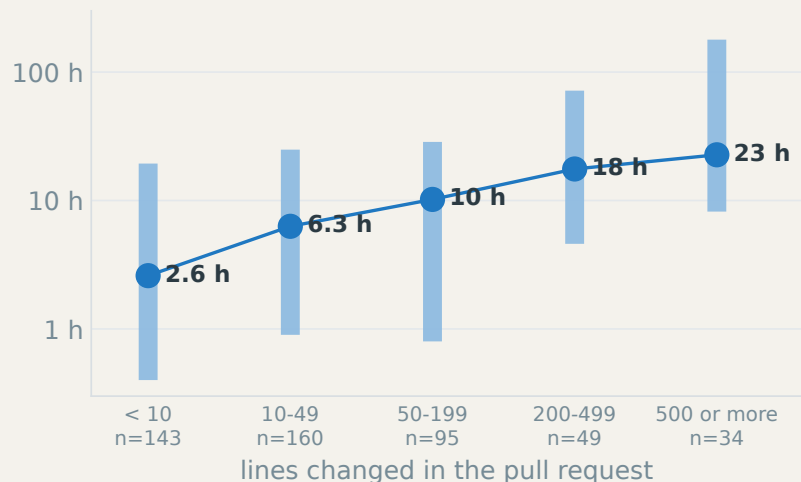


Share of first reviews, by quarter

- Two people give three quarters of first reviews. The top reviewer alone went from a third to over half.
- None of the 14 contributors new since January has given a first review yet.
- Design review is not in these numbers. ADRs are reviewed by everyone, in meetings and on chat.

→ *writing code has spread; reading it has not*

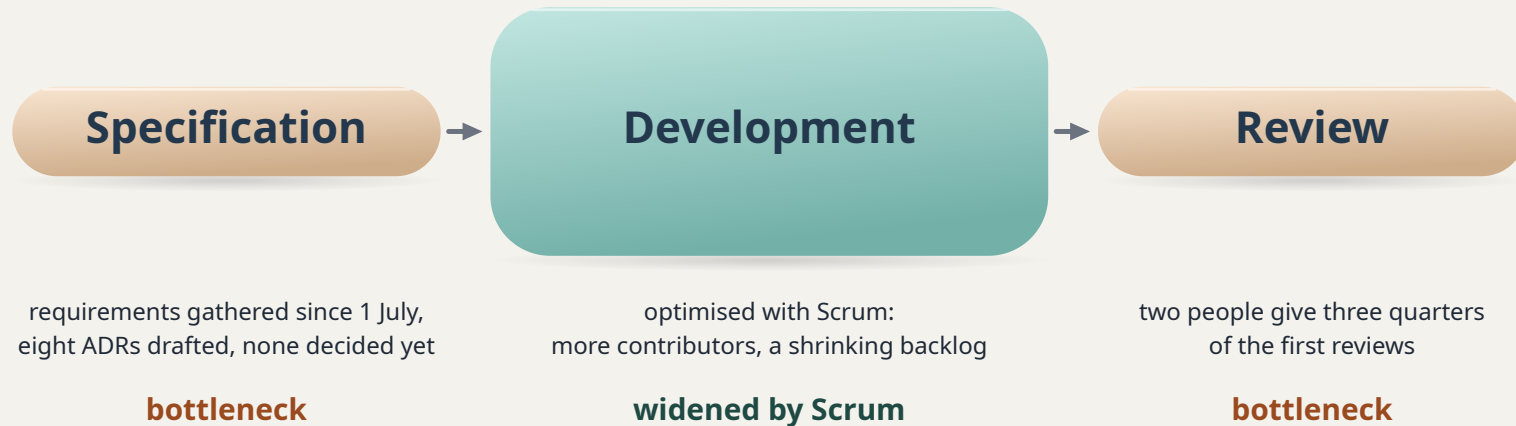
# Smaller pull requests get reviewed sooner



Median wait for a first review, with the middle half of the pull requests

- Under 10 lines: 2.6 hours. 500 lines or more: 23 hours. Time to merge goes from a few hours to five days.
  - The spread is wide at every size, so size helps but does not decide.
  - Who writes it matters too. For pull requests under 50 lines, the five core developers wait 2.5 hours; everyone else waits 11.
- *split the work, and announce a big change early*

# Where work waits



- **Scrum widened development:** more contributors, a shrinking backlog.
- **Specification and review are the narrow parts,** and they will matter more with every new component.

# | 5 · What's next

How we could improve, and the roadmap



# How you can help, and what we would like to ask

## Where a part-time contributor helps most

- **Review pull requests.** Our scarcest resource.
- **Port your plugins** (a catalogue feeder, a packer) when the transformation system lands.
- **Write a compute backend** (HTCondor, push to HPC).
- **Check the ADRs against your practice.** They lean toward LHCb today.

## Questions for developers and product owners

1. **Kanban style:** when your tasks are done, should you review before starting something new?
2. What would make you review a pull request from **another community**?
3. What should **the team own** that architects decide today, starting with the plan for the transformation system?

IDEAS FOR DEVELOPERS AND PRODUCT OWNERS TO WEIGH, NOT DECISIONS

finished your tasks? review before starting more (kanban style)

limit work in progress

split big changes, announce them early

named reviewers per component

time-boxed ADR reviews

the team owns its plan

# Agentic AI amplifies what the process already gets wrong

"AI does not resolve ambiguity. It amplifies it."

Nir Yechiel, [The bottleneck moved](#), July 2026

- Writing code got cheap. **Deciding what to build and checking it** did not.
- It does **not change our process**. It shows its weak points sooner.

## Each one is something we just saw

- **Vague specification** → verbose, AI-generated issues that go stale (an open action). A clear ADR with a scenario test is what an agent, or a newcomer, can implement.
- **Review bottleneck** → more code means more to read, by the same two people.
- **Unplanned work** → faster code does not shrink urgent maintenance.
- **A new kind of user** → DiracX speaks HTTP and OAuth2, so an agent can use it through MCP.

→ *invest in specifications, tests and review, not in typing*

# The roadmap moves with the communities

NOW → END 2026

## Prove the bridge, settle the design

- Future client for job status: load-test the new installation
- Resource status: DIRAC adapter (two pull requests open)
- Transformation ADRs: review, decision makers

THEN

## The transformation system, bottom-up

- CWL jobs become parcels, on a dummy backend
- Transformations, then workgraphs
- A `legacy-dirac` backend in parallel, so real runs start early

AFTER

## Real execution

- Data transformations through the RMS
- Native backends, and a share of work moved gradually
- One real production shadowed end to end

IN PARALLEL, AT THEIR OWN PACE

matchmaking proof of concept

interCEde

web application (to be largely rewritten)

MCP and agent interface

accounting and observability

*Priorities, not commitments. New requirements can reorder this page; the team writes the detailed plan.*

# If you run an installation today

## What you can count on

- **DIRAC keeps running**, with security patches and performance fixes when needed.
- **You choose when to switch each component**, usually at a shutdown.
- **No data migration** for anything but the transformation system.
- **Your extension keeps working** on DIRAC while you move.
- **Transformations move last and in steps**, with your DIRAC WMS still running the jobs.

## What we need from you

- **Tell us how many productions, plugins and workflows you have**, and which must survive. There is no compatibility with the new system, and nothing describes their migration yet.
- **Say which new transformation concepts do not fit** your practice.
- **Test in certification**; two installations already do.
- **Rename** `LegacyClientEnabled` to `FutureClientEnabled` if you set it.

# Take-aways

---

## FOR OPERATORS

- **Move to what exists as soon as you can; DIRAC is only patched.** You move one component at a time. The transformation system is the exception, and we plan it with you.

## FOR DEVELOPERS

- **Review is where the project is short.** New communities' requirements go through the same process as everyone's, and have already reordered the roadmap once.

## FOR THIS WEEK

- **Which productions and plugins must survive the move, and what would make you review?** Both change the plan.

# Thank you

Questions and comments welcome, this week and after.

**Alexandre F. Boyer** · [alexandre.franck.boyer@cern.ch](mailto:alexandre.franck.boyer@cern.ch)

[github.com/DIRACGrid](https://github.com/DIRACGrid) · [diracx.diracgrid.org](https://diracx.diracgrid.org)



# Backup: if you migrate extension code

1. **Run DIRAC 9** and a released DiracX (still 0.x: the API can change).
2. **Register all your VOs in DiracX** (configuration sync) *before* enabling anything.
3. **For each service you move:** build the DiracX route, write a `FutureClient` that implements *every* method of the legacy client, then switch it in the CS.
4. **Databases:** add tables beside DIRAC's, never alter an existing column.
5. **Use** `gubbins` as the template; if you extend routes, extend the client and regenerate it for each DiracX release.
6. **Leave alone** what you do not switch: agents, DISET services, pilots, the CS.

**Renamed on 3 September:** `LegacyClientEnabled` is now `FutureClientEnabled` (DIRAC  $\geq$  9.1.18). Rename it in your CS.

# Backup: what the RSS taught us

FEB 2026

RSS epic opened. User stories from a questionnaire. **No ADR yet.**

MAR – JUL

Database, logic and service written by **different people. The interfaces did not match.**

APR 2026

First ADR merged (tasks, DX-ADR-001).

JUN – JUL

Read routes released (v0.3.0). The DIRAC-side adapters are still open.

SEP 2026

Transformation ADRs 002–009 drafted.

## What we saw

- A synchronous cache met an asynchronous database.
- The legacy cache expected a VO on every row; the new API did not return one.
- Newcomers owned the change without a written design.
- **The plan was written for the team, not by the team.**

## What we do now

- **Design first:** an ADR before the code, reviewed by those who will use it.
- **Vertical tickets:** one person takes a behaviour from database to API, closed by a scenario test.
- **The team owns the plan.** Developers and product owners decide how the transformation system is cut and ordered.

# Backup: what each retrospective changed

## Done

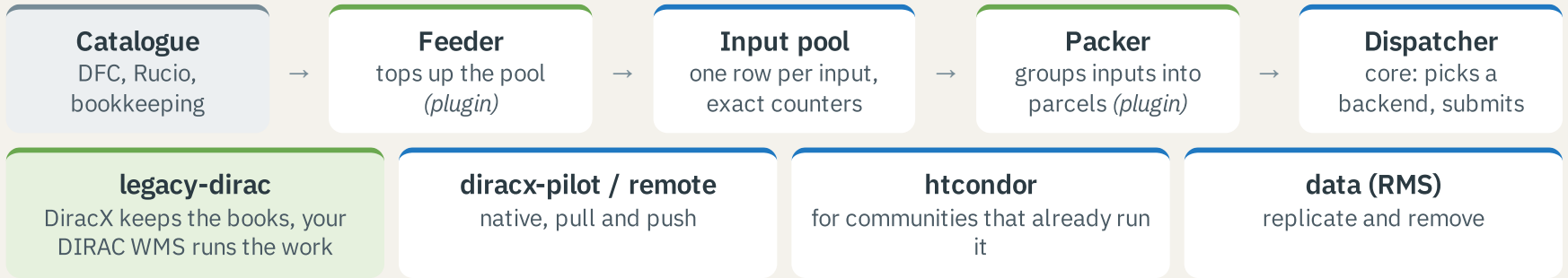
- ✓ Define what "done" means for a PR, and avoid new technical debt
- ✓ Do not plan dependent tasks in the same sprint
- ✓ Define the Scrum roles
- ✓ Estimates and velocity, with bonus points for external contributions
- ✓ Ask for reviews in the chat channel
- ✓ Announce a big PR early, and split the work

## In progress

- → Developers with spare time review PRs, at least a first pass
- → Architects share a more detailed roadmap
- → Reviewers double-check PR titles before merging
- → Feature PRs are tested in certification
- → Avoid verbose, AI-generated issues full of details that go stale
- → A better view of PRs ready for review versus needing changes

***Nine months in, the open items are mostly about review and about the roadmap.***

# Backup: what the new transformation system looks like



| IN DIRAC                     | IN DIRACX                                                                     |
|------------------------------|-------------------------------------------------------------------------------|
| Production                   | <b>Workgraph: a DAG of transformations, written in CWL</b>                    |
| Task                         | <b>Parcel: immutable, never retried (retries live on the input)</b>           |
| XML workflow, plugin classes | <b>CWL 1.2 with <code>dirac:</code> hints; feeder, packer, hooks, actions</b> |
| Integer TransformationID     | <b>UUID</b>                                                                   |

DX-ADR-002 to 009 are drafts, in a fork at the time of writing (decision makers TBD).

# Backup: the web application, and what we are weighing

## A · Stay: React + Next.js + MUI

- The 2024 plan: popular libraries, many developers
- Next.js now favours server rendering; we ship a static app
- July audit: hard to reuse outside its monorepo; extensions copy ~12 files that drifted

## B · Keep React, drop Next.js

- Vite plus a client-side router, a drafted proposal
- Smallest change; fixes the framework mismatch
- Keeps the dependency and upgrade treadmill

## C · Plain web components

- Browser standards, almost no dependencies, no build step
- An extension is a component you register, not files you copy
- Written with LLM help; **reviewed by people**

**Not decided.** C must prove: the job monitor stays fast on 500+ rows, accessibility and theming hold, and the review burden does not grow.

# Backup: where the numbers differ from the proceedings

- **Share of changes from outside the LHCb group**, 2025 / since January / since June: **19 / 42 / 54%** in the proceedings (merged pull requests, not de-duplicated); **20 / 42 / 56%** de-duplicated across release branches; **19 / 22 / 29%** if the GridPP developer is counted as core; **11 / 41 / 54%** if the three other LHCb contributors are counted as LHCb too.
- **Issue rates** (opened / closed a month, before and after January 2026): 16.3 → 27.3 and 10.6 → 36.6 in the proceedings; 16.7 → 27.6 and 10.9 → 37.6 recomputed. Not yet reconciled.
- **Top-two share of first reviews**: 61 → 75% (proceedings); 61 → 74% de-duplicated.
- **Window "since the last workshop"** starts 17 September 2025; the proceedings compare September 2024 – 20 January 2026 against 21 January – 4 September 2026.

# Backup: counting a community's contribution

| SINCE 17 SEP 2025                | MERGED PRS | LINES CHANGED (K) | PEOPLE | FIRST REVIEWS GIVEN | ISSUES OPENED | STORY POINTS |
|----------------------------------|------------|-------------------|--------|---------------------|---------------|--------------|
| LHCb core (5)                    | 403        | 158               | 5      | 499                 | 213           | 193          |
| Other LHCb                       | 13         | 13                | 4      | 1                   | 1             | 26           |
| CTAO                             | 49         | 34                | 7      | 11                  | 45            | 60           |
| GridPP                           | 109        | 10                | 3      | 9                   | 23            | 36           |
| IN2P3                            | 16         | 45                | 2      | 0                   | 3             | 33           |
| FCC                              | 9          | 39                | 2      | 8                   | 1             | 12           |
| EGI                              | 15         | 1                 | 2      | 0                   | 1             | 7            |
| CMS                              | 14         | 1                 | 6      | 0                   | 4             | 15           |
| Belle II · IHEP · not attributed | 6          | 0                 | 5      | 4                   | 9             | 25           |

Pull requests are what the project counts, but they hide size: GridPP's 109 changes are small SQL fixes, FCC's 9 and IN2P3's 16 are large. Story points: finished items since 21 January; 253 of 264 attributed (126 by assignee, 102 by the author of the pull request, 25 by the linked pull request). First reviews given by bots are excluded.